

Tuyển Tập Bài Tập: Căn Bậc Hai Trên Cấu Trúc Dữ Liệu & Mảng (Square Root Decomposition - SRD)

(16 Bài toán Tuyển chọn Phân cấp từ Codeforces, CSES, SPOJ & Luogu)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Kỹ thuật Phối hợp	Độ phức tạp Mục tiêu
1	CSES 1648 (Dynamic Range Sum Queries)	CSES Range Queries	Khởi động	Chia khối cơ bản Point Update & Range Sum	$O(Q\sqrt{N})$
2	Codeforces 13E (Holes)	Codeforces Beta Round 13	Khởi động	Sqrt Jump Pointers (Số bước nhảy ra khỏi khối)	$O((N + Q)\sqrt{N})$
3	SPOJ GIVEAWAY (Give Away)	SPOJ	Khởi động	Khối sắp xếp đếm số phần tử $\geq c$ trong $[a, b]$	$O(Q\sqrt{N \log N})$
4	Codeforces 455D (Sereja and Array / Sereja and Fun)	Codeforces Round 260	Khởi động	Khối Deque dịch chuyển xoay vòng đoạn con	$O(Q\sqrt{N})$
5	Codeforces 398D (Instant Messengers)	Codeforces Round 233	Trung cấp	Phân ngưỡng bậc Heavy-Light trên đồ thị bạn bè	$O(Q\sqrt{M})$
6	Codeforces 797E (Array Queries)	Educational Round 19	Trung cấp	Chia căn theo bước nhảy $k \leq \sqrt{N}$ vs $k > \sqrt{N}$	$O((N + Q)\sqrt{N})$
7	Codeforces 1207F (Remainder Problem)	Educational Round 71	Trung cấp	Chia căn theo số dư modulo $x \leq \sqrt{V}$	$O(Q\sqrt{V})$
8	Codeforces 1039D (You Are Given a Tree)	Codeforces Round 507	Trung cấp	Chia căn theo số đường đi tối đa $\leq \sqrt{N}$	$O(N\sqrt{N \log N})$
9	Codeforces 1580C (Train Maintenance)	Codeforces Round 745	Nâng cao	Chia căn theo chu kỳ bảo trì $x + y \leq B$	$O((N + M)\sqrt{M})$

STT	Mã Bài / Tên Bài	Nguồn / Nền tảng	Cấp độ	Kỹ thuật Phối hợp	Độ phức tạp Mục tiêu
10	Codeforces 342E (Xenia and Tree)	Codeforces Round 199	Nâng cao	Sqrt Rebuilding: Multi-Source BFS sau mỗi $\sqrt{Q}$ lần	$O((N + Q)\sqrt{N})$
11	Luogu P3396 (Hamster's problem / 哈希冲突)	Luogu	Nâng cao	Chia căn tổng bước nhảy chu kỳ mảng	$O(M\sqrt{N})$
12	Codeforces 1446D2 (Frequency Problem - Hard)	Codeforces Round 683	Nâng cao	Chia căn theo tần số phần tử lớn nhất	$O(N\sqrt{N})$
13	Codeforces 618F (Double Knapsack)	Wunder Fund Round 2016	Chỉnh phục	Nguyên lý Dirichlet + Mảng tiền tố chia căn	$O(N)$
14	Codeforces 911G (Mass Change Queries)	Educational Round 35	Chỉnh phục	Chia khối mảng hoán vị giá trị $1 \dots 100$	$O(Q\sqrt{N} + N)$
15	Luogu P6578 (Dashi's problem / 大力出奇迹)	NOI Online 2020 / Luogu	Chỉnh phục	Sqrt Decomposition kết hợp Bitset tăng tốc 64 lần	$O(\frac{N\sqrt{N}}{64})$
16	Codeforces 580E (Kefa and Watch)	Codeforces Round 321	Chỉnh phục	Chia khối Rolling Hash kiểm tra chu kỳ sâu	$O(Q\sqrt{N})$

1. Cấp Độ 1: Khởi Động & Nền Tảng (Warm-up & Fundamentals)

Bài 1: [CSES 1648 - Dynamic Range Sum Queries](#)

- **Nguồn:** [CSES Problemset — Range Queries](#).
- **Giới hạn:** $N, Q \leq 2 \times 10^5$, $X_i \leq 10^9$, Thời gian: 1.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Hỗ trợ hai loại truy vấn trên mảng A : (1) Gán $A[k] = u$; (2) Tính tổng các phần tử trong đoạn $[a, b]$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Bài toán nền tảng của SRD. Mảng được chia thành các khối kích thước $B = \lceil \sqrt{N} \rceil \approx 450$.
- **Bậc 2 (Cấu trúc dữ liệu):** Duy trì `block_sum[b]` là tổng các phần tử trong khối b .
- Cập nhật $A[k] = u$: `block_sum[k / B] += u - A[k]`, $A[k] = u$ trong $O(1)$.
- Truy vấn đoạn $[a, b]$: Tổng các khối nguyên ở giữa + duyệt tuần tự các phần tử lẻ ở khối a/B và b/B .
- **Bậc 3 (Chi tiết cài đặt):** Thời gian $O(1)$ cho cập nhật, $O(\sqrt{N})$ cho truy vấn. Sử dụng kiểu `long long` tránh tràn số.

Bài 2: [Codeforces 13E - Holes](#)

- **Nguồn:** [Codeforces Beta Round 13 - Problem E](#).
- **Giới hạn:** $N, M \leq 10^5$, Thời gian: 1.0s, Bộ nhớ: 64 MB.
- **Tóm tắt bài toán:** Có N cái hố. Khi một quả bóng rơi vào hố i , nó sẽ nhảy tới hố $i + power[i]$. Có hai loại truy vấn: (1) Thay đổi $power[i]$; (2) Thả bóng vào hố i , tìm số lần nhảy và vị trí hố cuối cùng trước khi bóng văng ra ngoài mảng.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Nếu nhảy từng bước, trường hợp xấu nhất tốn $O(N)$ cho mỗi truy vấn. Ta chia mảng thành các khối kích thước $B \approx \sqrt{N}$.
 - **Bậc 2 (Cấu trúc dữ liệu):** Với mỗi hố i , tiền xử lý:
 - `next_pos[i]`: Vị trí hố đầu tiên bóng chạm tới khi **vừa bay ra khỏi khối hiện tại** của i .
 - `jumps[i]`: Số bước nhảy cần thiết để từ i bay ra khỏi khối hiện tại.
 - **Bậc 3 (Chi tiết cài đặt):**
 - Khi tính toán trên khối: Duyệt ngược từ cuối khối về đầu khối để tính `next_pos[i]` và `jumps[i]` trong $O(1)$.
 - Khi cập nhật $power[i]$: Chỉ cần tính toán lại các hố trong cùng khối của i trong $O(B)$.
 - Khi truy vấn: Nhảy từ khối này sang khối khác bằng `next_pos`, mỗi bước nhảy bỏ qua trọn vẹn 1 khối $\implies$ mất tối đa $N/B = O(\sqrt{N})$ bước.
-

Bài 3: [SPOJ GIVEAWAY - Give Away](#)

- **Nguồn:** [SPOJ Problemset — GIVEAWAY](#).
- **Giới hạn:** $N \leq 5 \times 10^5$, $Q \leq 10^5$, Thời gian: 1.5s, Bộ nhớ: 1536 MB.
- **Tóm tắt bài toán:** Mảng N phần tử. Truy vấn (1) Đếm số phần tử trong $[a, b]$ có giá trị $\geq c$; (2) Cập nhật $A[a] = b$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Áp dụng mô hình **Khối sắp xếp (Sorted Blocks)**. Chia mảng thành các khối kích thước $B \approx \sqrt{N \log N}$.
 - **Bậc 2 (Cấu trúc dữ liệu):** Mỗi khối duy trì một `std::vector` đã sắp xếp.
 - Khi truy vấn: Các khối nguyên dùng `std::lower_bound` để đếm số phần tử $\geq c$ trong $O(\log B)$. Hai khối biên duyệt trâu trên mảng A .
 - Khi cập nhật: Gán $A[a] = b$, sau đó sắp xếp lại vector của khối chứa a trong $O(B \log B)$ hoặc xóa và chèn nhị phân trong $O(B)$.
 - **Bậc 3 (Chi tiết cài đặt):** Tổng thời gian $O(Q\sqrt{N \log N})$, chạy nhanh hơn rất nhiều so với Persistent Segment Tree về bộ nhớ và thời gian biên dịch.
-

Bài 4: [Codeforces 455D - Sereja and Fun](#)

- **Nguồn:** [Codeforces Round 260 \(Div. 1\) - Problem D](#).

- **Giới hạn:** $N, Q \leq 10^5$, Thời gian: 4.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Mảng N phần tử. Thao tác (1): Dịch chuyển xoay vòng đoạn $[L, R]$ sang phải 1 vị trí (phần tử $A[R]$ chuyển lên vị trí L); Thao tác (2): Đếm số lần xuất hiện của số K trong đoạn $[L, R]$. Bắt buộc giải trực tuyến (Online).

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Thao tác xoay vòng đoạn con rất khó thực hiện trên Segment Tree thông thường mà cần Treap. Nhưng với chia căn, ta có thể dùng `std::deque` cho từng khối!
- **Bậc 2 (Cấu trúc dữ liệu):** Chia mảng thành $\approx \sqrt{N}$ khối. Mỗi khối quản lý một `std::deque<int>` kích thước $\approx B$ và một mảng tần số `cnt[b][val]`.
- **Bậc 3 (Chi tiết cài đặt):**
 - Khi xoay vòng $[L, R]$: Xóa phần tử tại vị trí tương đối R trong khối của nó (pop ra), rồi chèn vào vị trí tương đối L trong khối của L . Sau đó cân bằng kích thước các khối ở giữa bằng cách đẩy phần tử cuối khối trước sang đầu khối sau trong $O(1)$ mỗi khối.
 - Khi truy vấn: Với khối nguyên, lấy ngay `cnt[b][K]`. Với khối biên, duyệt trực tiếp trên deque. Thời gian $O(Q\sqrt{N})$.

2. Cấp Độ 2: Kỹ Năng Trung Cấp (Intermediate Mastery)

Bài 5: [Codeforces 398D - Instant Messengers](#)

- **Nguồn:** [Codeforces Round 233 \(Div. 1\) - Problem D](#).
- **Giới hạn:** $N \leq 5 \times 10^4$, $Q \leq 2.5 \times 10^5$, $M \leq 1.5 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Mạng xã hội gồm các bạn bè nối với nhau. Các sự kiện: (1) Đổi trạng thái online/offline của một người; (2) Thêm/xóa quan hệ bạn bè; (3) Đếm số bạn bè đang online của người u .

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Áp dụng **Phân ngưỡng bậc Heavy-Light (Degree Thresholding)** với ngưỡng $B \approx \sqrt{M} \approx 400$.
- **Bậc 2 (Cấu trúc dữ liệu):**
 - Đỉnh nhẹ ($\deg < B$): Danh sách bạn bè nhỏ, khi hỏi trạng thái chỉ cần duyệt qua tất cả bạn bè để đếm ai đang online trong $O(B)$.
 - Đỉnh nặng ($\deg \geq B$): Số lượng đỉnh nặng tối đa $\leq 2M/B$. Mỗi đỉnh nặng duy trì một biến `online_friends_count`.
- **Bậc 3 (Chi tiết cài đặt):** Khi một đỉnh u chuyển trạng thái online/offline: u thông báo đến tất cả bạn bè là đỉnh nặng để tăng/giảm biến đếm của họ (vì số đỉnh nặng ít, tốn tối đa $O(\sqrt{M})$ bước). Khi hỏi u : nếu u nhẹ thì duyệt bạn bè, nếu u nặng thì in ra biến đếm trong $O(1)$.

Bài 6: [Codeforces 797E - Array Queries](#)

- **Nguồn:** [Educational Codeforces Round 19 - Problem E](#).

- **Giới hạn:** $N, Q \leq 10^5$, $A_i \leq N$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Mỗi truy vấn gồm cặp (p, k) . Lặp lại phép gán $p \leftarrow p + A[p] + k$ cho đến khi $p > N$. Tìm số thao tác cần thực hiện.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Nếu k lớn, số bước nhảy nhỏ. Nếu k nhỏ, số bước nhảy lớn nhưng số lượng giá trị k nhỏ lại ít. $\implies$ **Chia căn theo tham số k** với ngưỡng $B \approx \sqrt{N} \approx 320$.
- **Bậc 2 (Tiền xử lý & Trực tuyến):**
- Với $k \leq B$: Tiền xử lý bảng quy hoạch động `dp[p][k]` là số bước để nhảy từ p ra ngoài khi dùng tham số k . Chạy ngược từ N về 1: `dp[p][k] = 1 + dp[p + a[p] + k][k]`. Tốn $O(N\sqrt{N})$ thời gian và bộ nhớ.
- Với $k > B$: Mỗi bước p tăng ít nhất B , do đó số bước nhảy tối đa không vượt quá $N/B = O(\sqrt{N})$. Ta duyệt trực tiếp từng bước.
- **Bậc 3 (Chi tiết cài đặt):** Truy vấn trả lời trong $O(1)$ nếu $k \leq B$ và $O(\sqrt{N})$ nếu $k > B$. Tổng thời gian $O((N + Q)\sqrt{N})$.

Bài 7: [Codeforces 1207F - Remainder Problem](#)

- **Nguồn:** [Educational Codeforces Round 71 - Problem F](#).
- **Giới hạn:** $N = 500\,000$, $Q \leq 500\,000$, Thời gian: 4.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Mảng khởi tạo toàn 0. Có Q thao tác: (1) $A[x] += y$; (2) Tính tổng các phần tử $A[i]$ có chỉ số thỏa mãn $i \equiv y \pmod{x}$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Chia căn theo giá trị bước nhảy x với ngưỡng $B = \lceil \sqrt{N} \rceil \approx 700$.
- **Bậc 2 (Cấu trúc dữ liệu):**
- Duy trì bảng `ans[x][y]` lưu tổng các phần tử có $i \equiv y \pmod{x}$ với mọi $x \leq B, 0 \leq y < x$.
- Mảng gốc $A[i]$ kích thước N .
- **Bậc 3 (Chi tiết cài đặt):**
- Khi cập nhật $A[x] += y$: Cộng y vào $A[x]$ và cập nhật `ans[mod][x % mod] += y` cho tất cả $1 \leq \text{mod} \leq B$ (mất $O(B)$).
- Khi truy vấn (x, y) : Nếu $x \leq B$, lấy ngay `ans[x][y]` trong $O(1)$. Nếu $x > B$, duyệt qua các chỉ số $i = y, y + x, y + 2x \dots \leq N$ để cộng $A[i]$, tốn tối đa $N/B = O(\sqrt{N})$ bước.

Bài 8: [Codeforces 1039D - You Are Given a Tree](#)

- **Nguồn:** [Codeforces Round 507 \(Div. 1\) - Problem D](#).
- **Giới hạn:** $N \leq 10^5$, Thời gian: 7.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Cho cây N đỉnh. Với mỗi $k \in [1 \dots N]$, tìm số lượng đường đi đơn độ dài $\geq k$ nhiều nhất có thể chọn sao cho các đường đi không giao nhau ở đỉnh.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Với một giá trị k cố định, ta có thể tham lam (Greedy DP trên cây) từ dưới lên trong $O(N)$ bằng thứ tự duyệt DFS đảo ngược.
- **Bậc 2 (Chia căn đáp án):** Gọi $f(k)$ là đáp án với độ dài k . Ta có nhận xét $f(k) \leq N/k$.
- Với $k \leq B \approx \sqrt{N \log N}$: Tính trực tiếp $f(k)$ bằng DP cây, mất $O(B \cdot N)$.
- Với $k > B$: Giá trị của $f(k)$ chỉ có thể nằm trong khoảng $[0, N/B]$. Vì $f(k)$ là hàm không tăng, ta có thể dùng **chặt nhị phân** tìm khoảng các giá trị k liên tiếp có cùng kết quả $f(k)$.
- **Bậc 3 (Chi tiết cài đặt):** Mảng thứ tự duyệt DFS phẳng giúp khử đệ quy hoàn toàn, tối ưu tốc độ duyệt cây. Tổng thời gian $O(N\sqrt{N \log N})$.

3. Cấp Độ 3: Kỹ Năng Nâng Cao (Advanced Techniques)

Bài 9: [Codeforces 1580C - Train Maintenance](#)

- **Nguồn:** [Codeforces Round 745 \(Div. 1\) - Problem C](#).
- **Giới hạn:** $N, M \leq 2 \times 10^5$, Thời gian: 3.5s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Có N loại tàu hỏa. Tàu i hoạt động x_i ngày rồi bảo trì y_i ngày, chu kỳ $x_i + y_i$. Các sự kiện thêm tàu vào hệ thống hoặc rút tàu ra. Sau mỗi ngày, đếm số tàu đang bảo trì.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Chu kỳ $L_i = x_i + y_i$. Chia căn theo chu kỳ với ngưỡng $B \approx \sqrt{M} \approx 450$.
- **Bậc 2 (Cấu trúc dữ liệu):**
 - Tàu có chu kỳ nhỏ ($L_i < B$): Duy trì mảng `cnt[mod][rem]` là số tàu có chu kỳ `mod` đang bảo trì vào ngày có số dư `rem`. Cập nhật trong $O(L_i) \leq O(B)$.
 - Tàu có chu kỳ lớn ($L_i \geq B$): Số khoảng bảo trì trong M ngày là không quá M/B . Ta dùng một mảng hiệu (Difference Array) `diff[d]` để đánh dấu các đoạn bảo trì trong tương lai.
- **Bậc 3 (Chi tiết cài đặt):** Mỗi ngày, số tàu bảo trì bằng tổng tích lũy từ mảng `diff` cộng với $\sum_{m=1}^{B-1} cnt[m][day \pmod m]$. Tổng thời gian $O((N + M)\sqrt{M})$.

Bài 10: [Codeforces 342E - Xenia and Tree](#)

- **Nguồn:** [Codeforces Round 199 \(Div. 2\) - Problem E](#).
- **Giới hạn:** $N, M \leq 10^5$, Thời gian: 5.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cây N đỉnh. Ban đầu chỉ có đỉnh 1 được tô đỏ, các đỉnh khác màu xanh. Thao tác (1): Tô đỏ đỉnh v ; Thao tác (2): Tìm khoảng cách từ v đến đỉnh màu đỏ gần nhất.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Áp dụng **Sqrt Rebuilding**. Gom nhóm các truy vấn cập nhật thành từng batch kích thước $B \approx \sqrt{M} \approx 320$.
- **Bậc 2 (Cấu trúc dữ liệu):**
 - Mảng `dist[u]` lưu khoảng cách từ u đến đỉnh đỏ gần nhất được tính toán tại lần rebuild gần nhất.
 - Một vector `buffer` chứa các đỉnh vừa được tô đỏ trong đợt hiện tại (chưa được rebuild).

- **Bậc 3 (Chi tiết cài đặt):**
- Khi truy vấn v : Khoảng cách nhỏ nhất bằng $\min(dist[v], \min_{x \in buffer} dist(v, x))$. Tính $dist(v, x)$ bằng LCA trong $O(1)$ hoặc $O(\log N)$. Vì $|buffer| \leq B$, thao tác mất $O(B)$.
- Khi $|buffer| \geq B$: Chạy Multi-Source BFS từ toàn bộ các đỉnh đổ trên cây để cập nhật lại toàn bộ mảng `dist[u]` trong $O(N)$, sau đó xóa rỗng buffer. Tổng thời gian $O((N + M)\sqrt{M})$.

Bài 11: [Luogu P3396 - Hamster's problem / 哈希冲突](#)

- **Nguồn:** [Luogu Problemset — P3396](#).
- **Giới hạn:** $N, M \leq 1.5 \times 10^5$, Thời gian: 1.0s, Bộ nhớ: 128 MB.
- **Tóm tắt bài toán:** Mảng A . Truy vấn (1) Tính $\sum A[i]$ với $i \equiv y \pmod{x}$; (2) Sửa $A[x] = y$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Chia căn số dư theo ngưỡng $B = \lceil \sqrt{N} \rceil \approx 400$.
- **Bậc 2 (Cấu trúc dữ liệu):** Bảng `dp[p][k]` lưu tổng các phần tử có $i \equiv k \pmod{p}$ với $p \leq B$.
- **Bậc 3 (Chi tiết cài đặt):**
- Truy vấn: $p \leq B$ trả lời trong $O(1)$; $p > B$ bước nhảy lớn, duyệt thẳng trong $O(N/B) = O(\sqrt{N})$.
- Cập nhật: Thay đổi $A[x]$ thì cập nhật lại B giá trị trong bảng `dp` tốn $O(B)$.

Bài 12: [Codeforces 1446D2 - Frequency Problem \(Hard Version\)](#)

- **Nguồn:** [Codeforces Round 683 \(Div. 1\) - Problem D2](#).
- **Giới hạn:** $N \leq 2 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Tìm độ dài đoạn con liên tiếp dài nhất có ít nhất 2 phần tử cùng đạt tần số xuất hiện cực đại trong đoạn con đó.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Gọi M là phần tử có tần số xuất hiện nhiều nhất trên cả mảng. Chia căn dựa trên tần số xuất hiện với ngưỡng $B \approx \sqrt{N}$.
- **Bậc 2 (Chia 2 trường hợp):**
- Các phần tử có tần số $\geq B$: Số lượng tối đa N/B . Với mỗi phần tử X , ánh xạ mảng thành $+1$ (nếu là M) và -1 (nếu là X), tìm đoạn tổng 0 dài nhất trong $O(N)$.
- Các phần tử có tần số $< B$: Với mỗi giá trị tần số $f \in [1 \dots B]$, tìm đoạn dài nhất có ≥ 2 phần tử đạt tần số f bằng Two Pointers trượt.
- **Bậc 3 (Chi tiết cài đặt):** Cân bằng hai pha đạt độ phức tạp chuẩn $O(N\sqrt{N})$.

4. Cấp Độ 4: Thử Thách & Chinh Phục (Mastery & Advanced Variants)

Bài 13: [Codeforces 618F - Double Knapsack](#)

- **Nguồn:** [Wunder Fund Round 2016 - Problem F](#).

- **Giới hạn:** $N \leq 10^6$, $A_i, B_i \leq N$, Thời gian: 2.5s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Cho hai mảng A và B gồm các số nguyên dương $\leq N$. Tìm hai tập chỉ số con liên tiếp không rỗng của A và B sao cho tổng hai tập con bằng nhau.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Xét mảng tiền tố $P_A[i]$ và $P_B[j]$. Giả sử không mất tính tổng quát $P_A[N] \leq P_B[N]$.
- **Bậc 2 (Nguyên lý Dirichlet):** Với mỗi $i \in [0 \dots N]$, tìm j nhỏ nhất sao cho $P_B[j] \geq P_A[i]$. Khoảng cách $P_B[j] - P_A[i]$ nằm trong đoạn $[0, N - 1]$.
- **Bậc 3 (Chi tiết cài đặt):** Có $N + 1$ giá trị i nhưng chỉ có N hiệu số dư khả dĩ. Theo nguyên lý Dirichlet (nguyên lý chuồng bồ câu), tồn tại hai giá trị $i_1 < i_2$ có cùng hiệu số $P_B[j] - P_A[i]$, từ đó suy ra tổng hai đoạn bằng nhau. Độ phức tạp $O(N)$ bằng hai con trỏ.

Bài 14: [Codeforces 911G - Mass Change Queries](#)

- **Nguồn:** [Educational Codeforces Round 35 - Problem G](#).
- **Giới hạn:** $N \leq 2 \times 10^5$, $Q \leq 2 \times 10^5$, $A_i \leq 100$, Thời gian: 3.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Mỗi thao tác (L, R, x, y) thay thế tất cả phần tử có giá trị x thành y trong đoạn $[L, R]$. Giá trị phần tử chỉ từ 1 đến 100.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Vì dài giá trị rất nhỏ ($V \leq 100$), ta có thể chia mảng thành các khối kích thước $B \approx \sqrt{N}$.
- **Bậc 2 (Cấu trúc dữ liệu):** Với mỗi khối b , duy trì một mảng ánh xạ `map_val[b][v]` biểu diễn: giá trị gốc v hiện tại đang được đổi thành giá trị nào.
- **Bậc 3 (Chi tiết cài đặt):**
 - Khối nguyên: Chỉ cần cập nhật mảng ánh xạ: nếu `map_val[b][v] == x` thì gán lại thành y trong $O(V)$ cho mỗi khối.
 - Khối biên: Đẩy mảng ánh xạ xuống mảng A của khối, sửa đổi trực tiếp các phần tử lẻ, rồi khởi tạo lại `map_val[b][v] = v`.
 - Độ phức tạp: $O(Q \cdot (N/B \cdot V + B)) = O(Q\sqrt{N \cdot V})$.

Bài 15: [Luogu P6578 - Dashi's problem / 大力出奇迹](#)

- **Nguồn:** [NOI Online 2020 / Luogu](#).
- **Giới hạn:** $N, M \leq 3 \times 10^5$, Thời gian: 2.0s, Bộ nhớ: 512 MB.
- **Tóm tắt bài toán:** Cập nhật điểm và đếm số lượng đoạn con liên tiếp toàn số $\geq X$ trong đoạn $[L, R]$.

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Bài toán cực khó về tối ưu hằng số. Chia căn mảng kết hợp kỹ thuật nén bit `std::bitset` hoặc `unsigned long long`.

- **Bậc 2 (Cấu trúc dữ liệu):** Chia khối $B \approx \sqrt{N}$. Mỗi khối biểu diễn dãy nhị phân (1 nếu $\geq X$, 0 nếu $< X$).
- **Bậc 3 (Chi tiết cài đặt):** Tận dụng các phép toán bitwise 64-bit (`_builtin_clzll`, `_builtin_ctzll`) để tính nhanh các đoạn 1 liên tiếp trên ranh giới các khối, tăng tốc độ xử lý gấp 64 lần.

Bài 16: [Codeforces 580E - Kefa and Watch](#)

- **Nguồn:** [Codeforces Round 321 \(Div. 2\) - Problem E](#).
- **Giới hạn:** $N \leq 10^5$, $Q \leq 10^5$, Thời gian: 1.5s, Bộ nhớ: 256 MB.
- **Tóm tắt bài toán:** Xâu gồm các chữ số. Có thao tác gán cả đoạn $[L, R]$ thành chữ số c . Truy vấn: Kiểm tra xem xâu con $[L, R]$ có tuần hoàn với chu kỳ d hay không (nghĩa là $S[i] = S[i + d]$ với mọi $L \leq i \leq R - d$).

Gợi ý 3 bậc:

- **Bậc 1 (Nhận diện):** Đoạn $[L, R]$ có chu kỳ $d \iff$ Tiền tố độ dài $(len - d)$ trùng khớp với hậu tố độ dài $(len - d)$ của đoạn đó:

$$Hash(L, R - d) == Hash(L + d, R)$$

- **Bậc 2 (Cấu trúc dữ liệu):** Dùng Rolling Hash kết hợp Segment Tree Lazy hoặc Chia căn khối mảng.
- **Bậc 3 (Chi tiết cài đặt):** Thao tác gán cả đoạn thành chữ số c tương đương với việc gán mã hash của khối thành tổng cấp số nhân $c \sum B^i$, tính toán trong $O(1)$. So khớp hash trong $O(\log N)$ hoặc $O(\sqrt{N})$.



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.